



WEB班ワークショップ説明資料案1



今日のゴール

完成形のタスク管理アプリを段階的に再現して、CSS装飾とJavaScript動作の実装力を身につける



ワークショップの流れ

Step 0: 完成形を体験しよう（10分）

まずは今日作るアプリを実際に触ってみましょう！

やること:

1. 完成形のタスク管理アプリを操作
2. どんな要素・機能があるか観察
3. 「これを作るには何が必要？」を考える

発見してほしいポイント:

- 📄 ヘッダー部分
- 📊 進捗バー
- ✅ 6つのタスクチェックボックス
- ✨ マウスホバー時の動き
- 🎉 全完了時のメッセージ

Step 1: HTMLで骨組みを作ろう（30分）

🤔 まず考えよう（10分）

「完成形を作るのに必要なHTMLタグは？」

思考の流れ:

完成形で見たもの → HTMLタグ

└ タイトル部分 → <header>, <h1>

└ 進捗表示 → <section>, <div>

└ タスクリスト → <input>, <label>

└ 完了メッセージ → <div>

💻 実装しよう（20分）

今の目標: 見た目は気にせず、機能する骨組みを作る

重要なポイント:

- セマンティックなタグを使おう (header, section など)
- 後でCSSを当てやすいようにclass/id名を付けよう
- 進捗バーは「外側div + 内側div」の2重構造

この段階での見た目:

- ブラウザのデフォルトスタイル
- 装飾は一切なし
- でも機能はする!

よくあるつまづき & 解決法: **?** 「進捗バーってどう書くの?」 → `<div class="progress-bar"><div class="progress-fill"></div></div>`

? 「classとidの使い分けは?」 → class: 複数の要素に使う、id: 1つの要素だけ

Step 2: CSSできれいに装飾しよう (35分)

 違いを見つけよう (5分)

今のHTMLと完成形を比べて、「何が違う?」

発見するギャップ:

- 背景色がない
- カード型の白い背景がない
- 余白・間隔が足りない
- 影やフォントも違う

 段階的に装飾していこう (25分)

段階1: 全体の基本設定 (5分)

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box; /* 重要! */  
}
```

```
body {  
  background: #f3f4f6; /* グレー背景 */  
  font-family: sans-serif;  
}
```

段階2: カード型レイアウト (10分)

```
.main-header, section {  
  background: white; /* 白いカード */  
  padding: 2rem; /* 内側の余白 */  
  border-radius: 10px; /* 角丸 */  
}
```

```
box-shadow: 0 2px 10px rgba(0,0,0,0.1); /* 影 */
}
```

段階3: 進捗バー・タスクアイテム (10分)

- 進捗バーの見た目
- タスクアイテムのレイアウト (flexbox使用)

よくあるつまづき & 解決法: ? 「box-shadowの数値の意味は？」 → 水平位置 垂直位置 ぼかし 色 の順番

? 「rgbaって何？」 → 透明度付きの色指定 (rgba(0,0,0,0.1) = 薄い黒)

✅ 完成形チェック (5分)

静的な見た目が完成形と同じになったか確認！

Step 3: アニメーションで動きを付けよう (25分)

🔍 動きを観察しよう (5分)

完成形で「どこがどう動いていた？」

発見する動き:

- タスクにマウスを乗せると浮き上がる
- 進捗バーが伸びる
- 完了メッセージが表示される

✨ ホバーエフェクトを作ろう (15分)

考える順序:

- 「マウスを乗せた時」 = :hover
- 「なめらかに変化」 = transition
- 「浮き上がる」 = transform: translateY()
- 「影で立体感」 = box-shadow

実装例:

```
.task-item {
  transition: all 0.3s ease; /* なめらか変化の準備 */
}

.task-item:hover {
  background: #f1f5f9; /* 背景色変化 */
  transform: translateY(-2px); /* 2px上に移動 */
  box-shadow: 0 4px 12px rgba(79, 70, 229, 0.2); /* 影 */
}
```

よくあるつまづき & 解決法: ? 「:hoverは知ってるけど、何を変化させればいい？」 → 背景色、位置、影を組み合わせると立体感が出る

? 「transitionって必要？」 → ないとパツと変わって不自然。あるとなめらか

動作確認（5分）

完成形と同じアニメーションになっているかチェック！

Step 4: JavaScriptで動的機能を作ろう（20分）

動的機能を整理しよう（5分）

「クリックしたり操作すると何が起こる？」

実現したい機能:

- チェック → タスクの見た目変化（取り消し線）
- チェック → 進捗バー更新
- 全完了 → 完了メッセージ表示

段階的に実装しよう（15分）

段階1: 要素を取得（3分）

```
const checkboxes = document.querySelectorAll('.task-checkbox');
const progressFill = document.getElementById('progressFill');
const progressText = document.getElementById('progressText');
```

段階2: イベント処理（7分）

```
checkboxes.forEach(checkbox => {
  checkbox.addEventListener('change', function() {
    // チェック状態が変わった時の処理
    if (checkbox.checked) {
      // チェックされた時
    } else {
      // チェック外された時
    }
    updateProgress(); // 進捗更新
  });
});
```

段階3: 進捗計算・表示（5分）

```
function updateProgress() {
  const totalTasks = checkboxes.length; // 全タスク数
  const completedTasks = document.querySelectorAll('.task-checkbox:checked').length; // 完了数
  const percentage = Math.round((completedTasks / totalTasks) * 100); // パーセント計算

  // 進捗バー更新
  progressFill.style.width = percentage + '%';
  progressText.textContent = `${completedTasks} / ${totalTasks} 完了 (${percentage}%)`;
}
```

よくあるつまづき & 解決法:  「querySelectorAllって何？」 → 条件に合う全ての要素を取得（複数取得）

 「forEachって何？」 → 配列の各要素に対して同じ処理を実行

? 「:checkedって何？」 → チェックされているチェックボックスだけを選択



完成！

できるようになったこと

- ☒ HTMLで構造を設計する思考
- ☒ CSSでカード型レイアウト
- ☒ ホバーエフェクトの実装
- ☒ JavaScriptでDOM操作
- ☒ 進捗計算とリアルタイム更新

次のステップ

- 他のアニメーション効果を試してみる
- タスクの追加・削除機能
- ローカルストレージでデータ保存
- レスポンシブデザイン



困った時のサポート

デバッグの基本

1. **コンソールを確認**: F12 → Console
2. **console.log()を活用**: 変数の中身を確認
3. **段階的に確認**: 一つずつ動作確認

よくあるエラー

- ✗ 「要素が取得できない」 → HTML要素のclass/id名とJavaScriptが一致しているか確認
- ✗ 「スタイルが効かない」 → CSSのセレクタ名が正しいか確認
- ✗ 「JavaScriptが動かない」 → コンソールにエラーメッセージが出ていないか確認

質問のコツ

- 「〇〇がうまくいかない」ではなく
- 「〇〇を実現したいが、△△の部分で詰まっている」と具体的に